

ORIGINAL RESEARCH

Hybrid traffic scheduling in time-sensitive networking for the support of automotive applications

Hongrui Nie¹  | Yue Su² | Weibo Zhao² | Junsheng Mu¹

¹School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing, China

²Cloud Computing and Big Data Research Institute, China Academy of Information and Communications Technology, Beijing, China

Correspondence

Weibo Zhao, Cloud Computing and Big Data Research Institute, China Academy of Information and Communications Technology, Beijing, China.
Email: zhaoweibo@caict.ac.cn

Abstract

Time-sensitive networking (TSN) is considered one of the most promising solutions to address real-time scheduling in in-vehicle network due to its capabilities for providing deterministic service. The TSN working group proposed various traffic shaping mechanisms, while deterministic scheduling of hybrid traffic is still not effectively solved since the traffic requirements are difficult to satisfy by standalone or combined mechanisms with fixed time slot divisions. This article presents a time-aware multi-cyclic queueing and forwarding scheduling model, that integrates the no-wait enabled time-aware shaper and multi-cyclic queueing and forwarding shaping models. Then, a scheduling solution, dubbed “TSN scheduling optimizer” (TSO) is proposed that combines optimization methods and incremental techniques. TSO aims to balance the load to maximize flow schedulability while guaranteeing the service requirements of hybrid traffic. Simulation evaluations through OMNeT++ provide a performance assessment of this proposed scheduling model, which can satisfy multiple types of traffic transmission requirements. Furthermore, TSO is compared with other baseline scheduling solutions, and TSO shows efficacy regarding execution time and schedulability.

1 | INTRODUCTION

With the rapid development of the automotive field, various applications are widely employed, such as advanced driver-assistance systems (ADAS) and automated driving system (ADS) [1], thus continuously improving the corresponding intelligence as a consequence. Motivated by this, more sensors and electronic control units (ECUs) are required to fulfil the functional safety requirements therein. These sensors and ECUs are distributed and highly interconnected with the capacity of sensing, communication and processing, together to form in-vehicle networks (IVNs). IVNs always rely on real-time communication and processing of in-vehicle sensor information to improve vehicle perception in real-time and obtain comprehensive environmental information. Such applications require high data transmission rate, low latency, and tight coordination between sensing and communication to improve deterministic service.

Time-sensitive networking (TSN) is the technology-enhancing standard Ethernet that offers significant opportunities to support deterministic communication with

bounded latency and low jitters. As a result, TSN is actively promoted in IVNs by providing deterministic data transmission services at the data link layer based on standard Ethernet. According to the IEEE 802.1 TSN task group, TSN focuses on time synchronization, traffic shaping, resource management, and reliability to achieve real-time and deterministic transmission, as listed in Table 1. To enable the time-aware scheduling defined in 802.1Q [3], TSN standardizes the time synchronization protocol 802.1AS [2]. This protocol allows all devices in the TSN network to synchronize their local clocks. The frame replication and elimination for reliability defined in 802.1CB [4] is discussed for fault tolerance.

In TSN, the traffic shaper, deployed at the egress port of the device, aims at the filtering of incoming flows, and the filtered flows are placed into one of the eight priority queues thereafter. There are many shapers in TSN standards, which can be divided into hard real-time shapers and soft real-time shapers. The hard real-time shaper relies on the gate control list (GCL), which determines the opening and closing times of each queue gate, while soft real-time shaper does have synchronisation dependence. The IVN supports many real-time applications, such as

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2023 The Authors. *IET Communications* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

TABLE 1 TSN standards included in IEEE 802.1 working group.

Capability domain	Standards
Time synchronization	
Timing and synchronization	802.1AS-2020 (a profile of IEEE 1588)
Reliability	
Frame replication and elimination	802.1CB-2017
Path control and reservation	802.1Qca-2015
Per-stream filter and policing	802.1Qci-2017
Reliability for time sync	802.1AS-2020
Bounded latency	
Credit based shaper (CBS)	802.1Qav
Frame preemption	802.3br-2016 /802.1Qbu-2016
Scheduled traffic	802.1Qbv-2015
Cyclic queuing and forwarding	802.1Qch-2017
Asynchronous traffic shaping	P802.1Qcr
Shaper parameter setting	P802.1Qdq
QoS provisions	P802.1DC
Resource management	
Stream reservation protocol	802.1Qat-2010
Path control and reservation	802.1Qca
Link-local registration protocol	802.1CS
TSN configuration	802.1Qcc
Foundational bridge YANG	802.1Qcp
YANG for CFM	P802.1Qcx
Extended stream identification	P802.1CBdb
Resource allocation protocol	P802.1Qdd

safety alarms and control messages from the driver assistant system, which have stringent quality of service (QoS) requirements. Thus, the hard real-time shapers are well suited for IVN.

Unlike the hardware functionality, routing and scheduling solutions have been left out of TSN standards. The configuration issues of the time-aware shaper (TAS) have been extensively studied in the literature [5–8], while the time cost will be unacceptable with the large network size and traffic scale. Hence, the asynchronous mechanism credit-based shaper (CBS) defined in IEEE 802.1BA [9] is introduced to prevent the starvation of class B traffic. Furthermore, TAS+CBS [10] can guarantee zero jitters for time-triggered (TT) traffic and settle the uncertainty of queuing for stream-reservation traffic.

To simplify the configuration and keep the latency of the traffic within a range, the IEEE 802.1Qch standard designs cyclic queuing and forwarding (CQF) shapers [11]. CQF is well suited for real-time applications with loose requirements for delay and jitter boundaries. The CQF is implemented by two queues that alternate between incoming and outgoing queue operations. Yan et al. [12] proposed the Tabu-ITP algorithm regulating the injection time for each flow, which greatly improved the scheduling success rate by randomly swapping the set of scheduled flows with the set of failed flows. However, deciding on a proper cycle is difficult in hybrid traffic scheduling since the

end-to-end delay of a flow is dependent on the number of routes and the cycle. Generally, large cycles lead to high latency, and small cycles require high bandwidths. However, using the same cycle for traffic with different periods and deadline requirements may result in a large number of non-schedulable flows. Thus, CQF alone cannot support hybrid traffic transmission.

Additionally, Finn [31] proposed multiple cyclic queuing and forwarding (Multi-CQF) as an extension of CQF. Konstantinos [32] proposed a multi-CQF configuration using a constraint programming formulation and simulated annealing-based meta-heuristic solution. It proved that multi-CQF could handle well flows even with tight timing constraints. However, as the network size increases, CQF and multi-CQF shapers will lead to larger latencies. In such situations, TAS is required for scheduling the most demanding streams. As [28] shows, there are many advantages to the combinations of shapers. However, there has been no systematic investigation of the combination of TAS and multi-CQF, as well as the configuration of TAMCQF.

To this end, we address the following research question in this article: (1) *How to design the scheduling model to satisfy the real-time requirements of different applications in the TSN network?* (2) *How to compute scheduling configurations under such schedule model for TSN?* We design the scheduling model integrating the TAS and multi-CQF shapers (TAMCQF) to support hybrid traffic transmission in TSNs. By assigning the offset for each flow based on different time granularity, the proposed model can guarantee the boundary end-to-end latency of audio video bridging (AVB) traffic without affecting TT traffic. To tackle the TSN scheduling configurations based on the TAMCQF model, we propose a TSN scheduling optimizer (TSO) combining optimization methods and incremental techniques depending on the resource requirements of the traffic as shown in Table 3. The optimization method is used to seek optimal offset decisions for TT traffic while the number of mapped AVB traffic is maximized by incremental techniques so that configuration results can be obtained more quickly. The contributions of this article are summarized as follows:

- **TAMCQF model:** We design a TAMCQF model integrating the TAS and multi-CQF shapers to simultaneously support hard real-time traffic, soft real-time traffic, and best-effort traffic mixed transmission. We further compare the proposed model with prevalent models (TAS+CBS, CQF, TAS+CQF) under the same settings using OMNeT++.
- **Scheduling model:** We formally define and model the configuration problem as the mixed integer linear programming (MILP) problem under the TAMCQF model. By assigning the offset for each traffic, TAMCQF model can guarantee zero jitter for TT traffic and low bounded end-to-end latency of AVB traffic without affecting TT traffic.
- **Optimized resource scheduling algorithm:** We propose the TSN scheduling optimizer (TSO) to tackle the TAMCQF based resource scheduling problem, which can attain high scheduling scalability through different cycle divisions, optimization methods, and incremental techniques. We compare

TABLE 2 Summary of notations.

Notation	Definition
Topology	
G	Network topology graph
V	The set of network devices (end systems and switches)
$v_a \in V$	Network device
E	Set of links
$\epsilon_{a,b} = (v_a, v_b)$	A physical link in the network
$\epsilon_{a,b} \cdot R$	Link bandwidth
Traffic parameters	
$P \in \{TT, AVB, BE\}$	Priority class of traffic (time-triggered, audio video bridging and best effort)
F_P	The set of flows with priority class P
$f_k^P, k \in \{1, \dots, F^P \}$	A flow at the priority class P
Scheduling	
T_{GCL}	The hyper-period of gate control list
$P.b$	Bandwidth allocated to priority class P
T_m	The minimum time slot in time-aware shaper scheduling
$T_c^{class, class} \in \{A, B\}$	The scheduling cycle of AVB- <i>class</i> queue
$\Phi[k, e] \in f_k^{TT} \cdot \Phi$	The offset of TT traffic f_k^{TT} on the link e
$f_k^{AVB} \cdot \phi$	The flow injection time slot of f_k^{AVB} at the sender
ω	The transmission window offset vector
R	The route decision vector

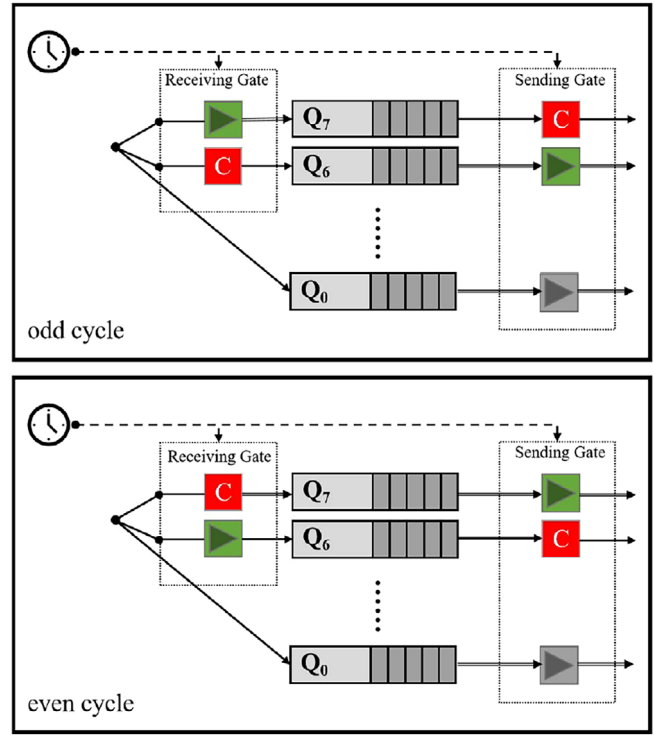
the proposed approach with other prevalent approaches (MILP optimal, the start-slot assignment [SSA] solution) under various settings, exhibiting the superiority of the proposed schemes.

The remainder of the article is structured as follows: The background and related work are discussed in Section 2. Section 3 discusses the TSN model under consideration, consisting of the network architecture, the scheduling model, and the traffic model. The scheduling problem formulation based on TAMCQF is presented in Section 4. The hybrid traffic scheduling solution based on TAMCQF is introduced in Section 5. Section 6 evaluates the proposed model and TSO solution. Finally, we conclude this article in Section 7.

2 | BACKGROUND AND RELATED WORKS

2.1 | Multi-CQF overview

CQF was proposed by the IEEE 802.1Qch [11], which utilized two queues and used receiving gate and sending gate to control the receive and send beats of the queue, respectively. The queues executed the enqueue and dequeue mechanism, which

**FIGURE 1** Schematic representation of CQF operations.

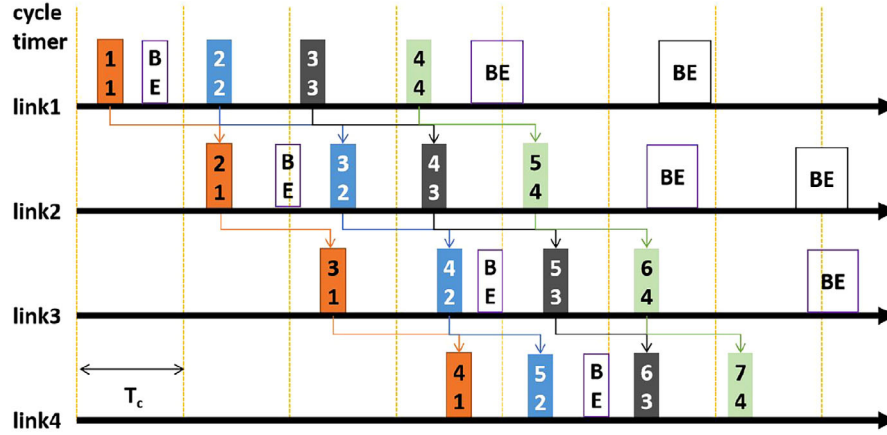
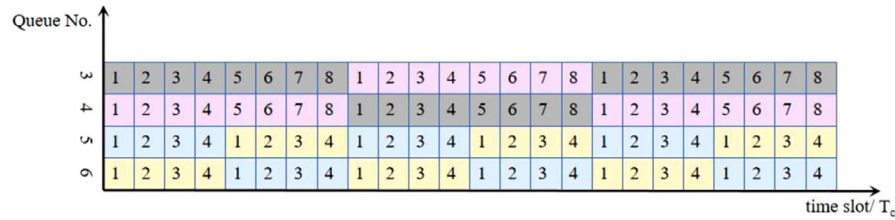
was also named Ping-Pong queues, as shown in Figure 1. In the odd cycle, the odd queue Q_7 opened the receiving gate to receive traffic from upstream nodes, and the even queue Q_6 opened the sending gate to transmit the frames to the downstream node. In the even cycle, the operation was executed conversely.

As shown in Figure 2, all outputs of the switches were implemented with the configured ping-pong queues with a pre-determined time cycle T_c . Time was divided into numbered slots with the length T_c . The frame was transmitted and queued along its path from link 1 to link 4 slot after slot. Once the flow was injected into the TSN network, its end-to-end latency was bounded in $[(H-1) \times T_c, (H+1) \times T_c]$, where H was the hops of the path. CQF could not avoid the maximum jitters of about $2T_c$. Hence, an appropriate value of time cycle T_c was one of the keys to affecting the performance of the CQF model. It could be difficult to pick a single value of T_c for a network. If T_c was small, then only a small amount of traffic could be accommodated in one cycle on one port, and the computation complexity of GCL was still high. If T_c was large, then more traffic could be accommodated, with a wide variation in allocated bandwidth. While the larger T_c would lead to high per-hop latency.

The multiple values of T_c for a single output port were proposed, as illustrated in Figure 3. The example had a schematic timeline. It ran two values of T_c at the same time. Q_5 and Q_6 used fast cycles ($T_{C_5} = T_{C_6} = 4 \times T_c$, where T_c is the minimum time slot unit in the CQF queue) with higher priority, while Q_3 and Q_4 ran with priorities 3 and 4 and used two times slower cycles than T_{C_5} and T_{C_6} . The numbers in Figure 3 mark the minimum time slot index included during the cycle. As an example,

TABLE 3 A classification of different traffic with the features and requirements.

Application name	Traffic class	Periodicity	Typical period	Typical applications	Jitter requirements	Typical data size
Isochronous	TT	Periodic	100 μ s–10 ms	Control data (security related)	0–1 μ s	Fixed:64–100 bytes
Cyclic	TT	Periodic	2–20 ms	Multimedia data (security related), Network control	≤ 1 ms	Fixed:64–1000 bytes
Audio/Voice	AVB	Periodic	Sampling rate	Voice interaction system (security-independent)	≤ 300 ms	Variable:1000–1500 bytes
Video	AVB	Periodic	Frame rate	Video command system	≤ 300 ms	Variable:1000–1500 bytes
Best effort	BE	Sporadic	Null	Upgrade log, Data logging	Null	Variable:30–1500 bytes

**FIGURE 2** Example of flow sequences based on the cyclic queuing and forwarding (CQF) mechanism.**FIGURE 3** Example of multiple T_c values in a multi-cyclic queuing and forwarding mechanism.

in the case of Queues 5 and 6, in the odd cycle (i.e. the $(2n + 1)$ st cycle), the data frames cached into the odd queue Q_5 will be forwarded to the next node (i.e., shown in the yellow part of the figure), and will be placed into the even queue Q_6 after entering the next switch port. And vice versa in the even cycle case. Note that higher-priority queues use a smaller cycle value, which means that the buffers can only hold less traffic each cycle. This is because the T_{c_q} value and the queue buffer length are mutually constrained, and the period of traffic in the queue must be divisible by the corresponding T_{c_q} . The mechanism of CQF enables that the range of end-to-end latency is related to T_{c_q} and the number of routing hops, that is, with the same traffic and network configuration, a lower latency can be attained by using a smaller cycle configuration. In accordance with the principle of strict priority, low-priority flows are interfered with by frames in high-priority, same-priority queues. However, the basic principle of CQF is still observed, that is, the total bandwidth of the link is not oversubscribed by the traffic, thus, each cycle (fast high-priority and slow low-priority) needs to be guaranteed to be able to transmit the data frames in its buffer for the duration of its cycle.

ple of CQF is still observed, that is, the total bandwidth of the link is not oversubscribed by the traffic, thus, each cycle (fast high-priority and slow low-priority) needs to be guaranteed to be able to transmit the data frames in its buffer for the duration of its cycle.

2.2 | Scheduling solution overview

The research on scheduling problems in TSN has expanded from various aspects, including scheduling solutions, and joint routing and scheduling planning.

In past years, many scheduling solutions under different scheduling models of TSN have been proposed. The TSN scheduling problem was formulated as an optimization problem with numerous forms. SMTs [14] solver was essentially a

form of backtracking searching to find feasible solutions to a constraint satisfaction problem. The constraints were the TSN topology and scheduled traffic. The formal-based scheduling methods of TSN IEEE802.1Qbv specifications were proposed in [15] and [16] based on an SMT formulation. The advantage of SMT was that the SMT solver was freely available to efficiently search for variable assignments that satisfied the Boolean expressions, while such expressions would take up a large amount of storage space compared to vectors and matrices. The same problem had been solved using metaheuristics [17, 29] that allowed for efficient schedule calculation. In [18], the authors proposed a solution to determine the routes of TT traffic for TSN-based networks using *integer linear programming (ILP)*. A mixed integer linear programming (MILP)-based scheduling approach was proposed in [19]. The proposed adaptive schedule based on MILP could react quickly to the changes during runtime. The article [20] addressed how to compute no-wait schedules and multipath routings for large-scale time-sensitive networks based on the ILP. The heuristic algorithm was an effective solution to tackle complicated computation problems in ILP solutions. An advanced earliest window heuristic scheduling algorithm was proposed to satisfy all the constraints of scheduling problems in multi-period, multi-queue, and multi-hop scenarios [21]. Bhattacharjee et al. [22] proposed a simulated annealing - based meta-heuristic to solve the joint placement, scheduling, and routing optimization problem.

The joint problem of routing and scheduling has also been investigated for TSN networks. The article [23] developed a novel joint traffic routing and scheduling algorithm to compute routes for TT traffic and construct the schedules. The joint routing and scheduling problem for multicast time-sensitive traffic based on ILP was formulated in [24, 25]. The configuration of routing and scheduling problem [26] was likewise investigated based on the CQF model. And the article [27] considered that the routes were given and focused on the assignment of flows to queues in the CQF model. Nowadays, hybrid scheduling for TSN-based IVN draws attention, and traffic shaping and approaches for solving scheduling configurations have actually made progress. However, hybrid flow scheduling mainly faces the following two challenges. One major challenge is how to ensure that both TT traffic together with AVB traffic, and best-effort (BE) traffic meet their QoS requirements at the same time. Another important challenge is that the complexity of solving the scheduling model or queue model based on the combinations of shapers is high, and an efficient optimization approach is required to solve and utilize the available network resources. Although the combined shaper is analysed at the theoretical level [28] that can effectively solve the problem of scheduling hybrid traffic, no corresponding scheduling configuration model and optimization solution method are available.

3 | TSN MODEL

In this section, we briefly describe the system model, consisting of the network architecture, the TSN switch model with the designed scheduling model, and the traffic model to build

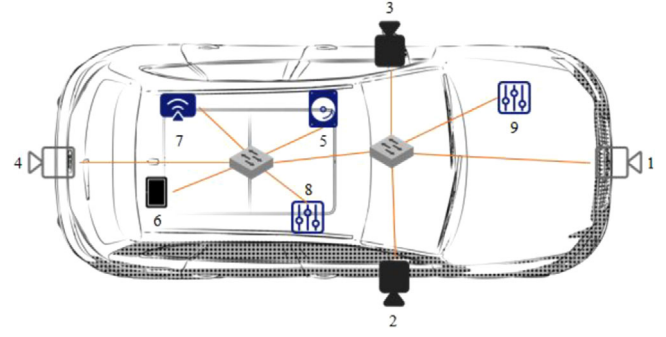


FIGURE 4 Simple schematic representation of the advanced driving assistance system domain network framework containing nine functional modules.

the envisioned traffic scheduling solution. All notations in this paper are listed in Table 2.

3.1 | Network architecture

The scenario considered in this work is shown in Figure 4. Nodes 1 to 4 represent cameras or sensors, and Node 5 is the Audio-CD, which transmits high-quality video and audio traffic to Node 6 (i.e. rear seat entertainment (RSE) system). Node 7 (i.e. telematics subsystem) transfers information (e.g. GPS data, traffic alerts) to Node 6. Node 8 is the application receiver, which elaborates, processes, and analyses all traffic from Node 1 to Node 5, and sends the warning and reports to Node 9. Let us consider a TSN as described above. The network consists of a set of end systems (ES) and switches (SW), also named nodes, connected by physical links. The topology of the network is modeled as a directed graph $G = (V, E)$, where $V = ES \cup SW$ is the set of all nodes in the TSN network, $E \subseteq V \times V$ indicates that there is a full-duplex link between two nodes connected in the network, which can be represented by a logical link in both directions (v_i, v_j) and (v_j, v_i) , where the first element denotes the source end system and the second element denotes the target end system. $|V|$ and $|E|$ are denoted as the number of nodes and the number of links in the network, respectively. Without loss of generality, we assume that all physical links have the same data rate BW in this paper. TSN networks can be multi-hop, where the output port of one SW is connected to the input port of an ES or another SW. The ADAS system includes two switches and nine end systems, and each end system is identified with an application. Each application can be split into various types of traffic, characterized by their priority level.

3.2 | The designed hybrid scheduling model

The designed hybrid scheduling model is depicted in Figure 5, where each port has 8 queues, from queue 7 to queue 0. Q_7 is the highest priority queue, which is used for TT traffic, CQF queues are from Q_6 to Q_3 , and the queues from Q_2 to Q_0 are used for

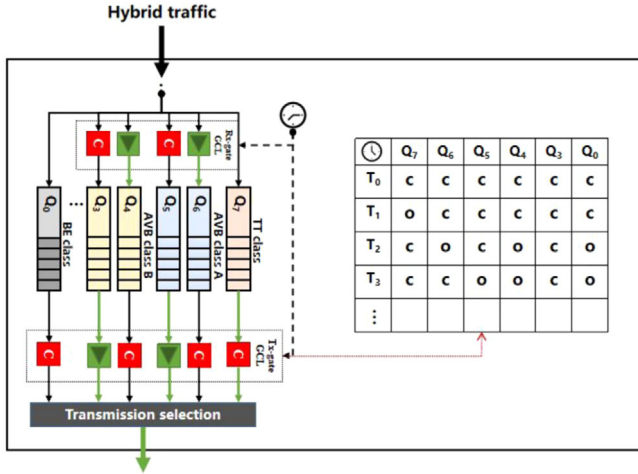


FIGURE 5 The designed hybrid scheduling, which integrates no-wait enabled time-aware shaper and the multi-cyclic queuing and forwarding shaper. In the gate control list, the “c” denotes closed, and the “o” denotes open.

alternative emergency traffic and best-effort (BE) traffic. Each queue is equipped with a timed gate, and all gates are controlled by gate control lists (GCLs). AVB traffic applies a multi-CQF shaper and occupies four queues. Each CQF queue owns its cycle length, and one grouping of CQF queues shares the same cycle and bandwidth.

In order to obtain effective and optimal scheduling results, it is necessary to precisely determine the forwarding time of TT traffic at each node and the time slot of AVB traffic to be sent at the source node, and determine the corresponding queue time slot resources do not overflow on the subsequent path, as shown in Figure 6. Generally, more bandwidth is required for traffic with a larger size and a higher period. If they are concentrated to be transmitted in the network in a time slot, then the network bandwidth will not be enough, which can lead to traffic loss or high latency. When the number of nodes in the network increases, the end-to-end delay range of the traffic increases with the rise in path length, so advance planning is required to prevent timeout situations. The remaining idle resources reserved for AVB traffic can be employed by BE traffic at any time, and the efficiency of resource utilization can be improved as far as possible in consequence. In addition, the time slot cycle of each traffic class can be adaptively determined based on the traffic period set, which can reduce the search space while accommodating more period types.

3.3 | Traffic model

Different nodes in Figure 4 have different transmission requirements. The application can be split into frames, which are sent from the ES via a flow with a single source and possibly multiple destinations. Each source ES is capable of sending multiple flows to the network. Our TSN solution supports three traffic types: TT traffic, AVB traffic, and BE traffic. We assume that the traffic type for each application has been determined by the

designer. TT traffic is small in volume, short in packet length, and requires very low latency and jitter for hard real-time applications. We assume that we have calculated the GCL of each node's output port in the priority queue Q_7^b , that is, the opening and closing times of TT traffic gating (called TT windows) and the GCL cycle T_{GCL} . Each TT flow f_k^{TT} can be described in Equation (1).

$$f_k^{TT} = \{f_k^{TT}.P, f_k^{TT}.class, f_k^{TT}.src, f_k^{TT}.dest, f_k^{TT}.size, f_k^{TT}.prd, f_k^{TT}.ddl, f_k^{TT}.route, f_k^{TT}.\Phi\} \quad (1)$$

where $f_k^{TT}.P$ denotes the traffic priority and all TT traffic is set to seven. $f_k^{TT}.class$ is the traffic class, $f_k^{TT}.src, f_k^{TT}.dest$ are the TT flow's source and destination, respectively. $f_k^{TT}.size$ defines the frame size, $f_k^{TT}.prd$ is the flow period and $f_k^{TT}.ddl$ is the deadline. $f_k^{TT}.route$ denotes the transmission path and $f_k^{TT}.\Phi$ denotes the time slot offset, which is configured by the results of the routing algorithm and the scheduling algorithm. AVB traffic with long periods, more quantity packets, and large packet sizes requires bounded latency. The AVB traffic can be classified into Class A and Class B according to their minimized sending interval, which is equal to the latency thresholds of each class [35]. Each AVB flow f_k^{AVB} is defined by an eight-tuple, similar to TT flow. While $f_k^{AVB}.\phi$ indicates flow injection time slot, which is different from the concept of time slot unit in the TT no waiting scheduling mechanism and differentiates with markers. BE traffic is a sporadic flow without time-bound QoS requirements. The scheduling of the BE flow does not need to be strictly planned, hence $f_k^{BE} = \{f_k^{BE}.P, f_k^{BE}.class, f_k^{BE}.src, f_k^{BE}.dest, f_k^{BE}.size, f_k^{BE}.route\}$.

The flow classification procedure can be performed in the manner shown in the Figure 7. Traffic within an in-vehicle network is classified based on real-time requirements. For example, the transmission requirements and traffic features from different functional domains are shown in Table 3 [33, 34]. Safety-related control data such as brake commands are referred to as isochronous traffic, which has stringent requirements for latency and jitter. The data frames contain essential control commands and are generally small. Sensors such as radar and ultrasonic need to capture the details of the surrounding environment in high resolution, thereby the data frame size will be larger. The sensing data needs to be synchronized to the central controller in real time to provide real-time feedback. This periodically reported traffic is referred to as cyclic traffic, which has loose requirements for latency and jitter. The above two types of traffic are classified as TT traffic in IVN. Audio and video traffic require smooth playback, which has loose jitter requirements while still require real-time mechanism to ensure user experience. Typically, audio and video traffic are classified as AVB traffic, which can be further divided into AVB-A and AVB-B class [35]. Such as data logging applications have low real-time requirements and do not require strict quality-of-service guarantees, which are classified as BE traffic.

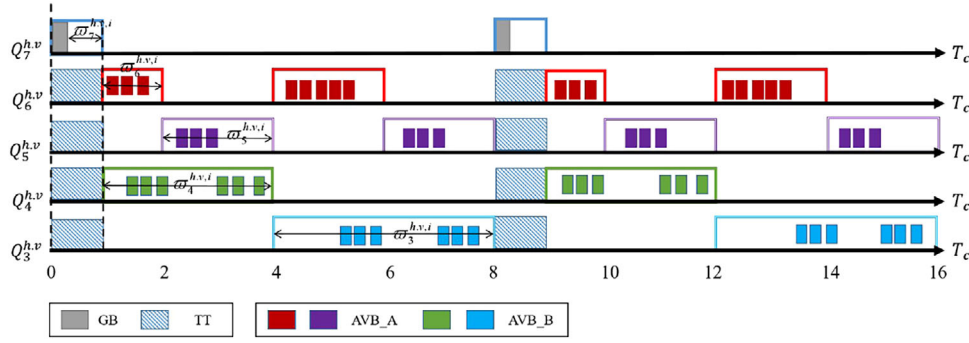


FIGURE 6 Possible scenario for the scheduling results of TSN switch.

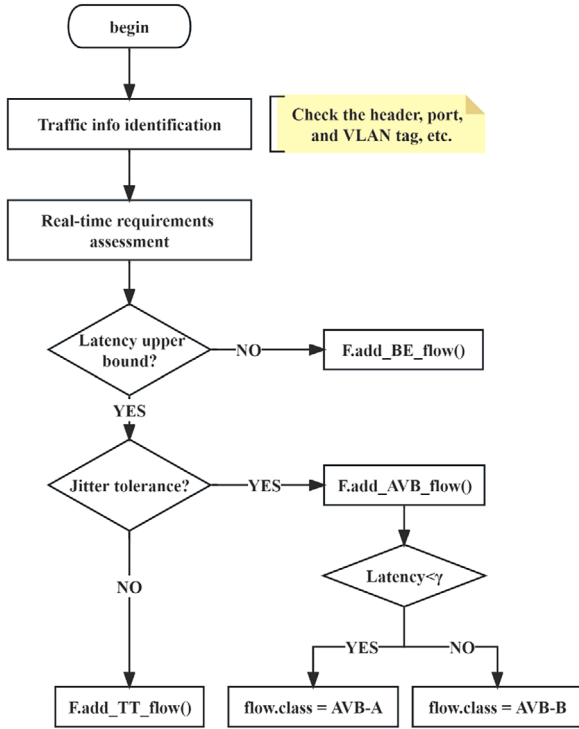


FIGURE 7 Traffic classification procedure.

4 | TRAFFIC SCHEDULING IN PROBLEM STATEMENT

4.1 | Decision variables

The decision-making variables for traffic scheduling are considered as follows.

- *Time slot offset*: We use $f_k^{TT} \cdot \Phi$ to describe the time slot offset vector in the first cycle with respect to the zero baselines on the corresponding port. It is a series of integer variables that indicates the transmission of the first frame belonging to flow f_k^{TT} starts at the time slot at each node.
- *Flow injection time slot*: $f_k^{AVB} \cdot \phi$ describes the offsets of injection time slot at the sender, on the other hand, the switch cannot adhere to this rule, that is, frames are forwarded hop by hop

based on the cycle. It is an integer variable related to the flow period.

4.2 | Scheduling constraints

If the worst-case end-to-end delays (WCDs) of the flow are smaller or equal to its deadline, we define that the flow is schedulable. The TAS model implements a completely deterministic transmission to achieve zero jitters, the latency is available once the GCL is constructed. We adopt a no-wait transmission constraint for TT traffic in the TAS model in this paper. And the TT traffic scheduling needs to solve how to generate the compact TT window shown in Figure 8. There are two methods to combine TT and AVB traffic transmissions. It is noticed that more available bandwidth can be saved for AVB traffic when TT windows are configured as compact back-to-back style and only one guard band is required during one cycle. We propose a modified no waiting scheduling constraint based on [29].

Especially, important goal referred to in the compact back-to-back style is to minimize the sending start time of the flow. In addition to bandwidth utilization, we also consider load balancing to avoid traffic aggregation on a limited link and time slot resource blocks. The approach solves the hybrid scheduling problems of TT traffic and AVB traffic independently with different objective optimization. The scheduling optimization objective of TT traffic is to minimize the forwarding starting time compared to the starting of the nearest cycle for overall TT flows and important window constraints are as follows. And another target of the scheduling is to map as many AVB applications as possible.

4.2.1 | TT window constraints

The offset of the data frame should be within the pre-defined TT window, which helps achieve a compact back-to-back style.

$$\forall f_k^{TT} \in F^{TT}, \forall e \in f_k^{TT} \cdot route : \quad (2)$$

$$\frac{L_{GB}}{R} \leq \Phi[k, e] \% T_c \leq \theta \lfloor \frac{\Phi[k, e]}{T_c} \rfloor \times T_c - \frac{l_{TT}^{max}}{R}$$

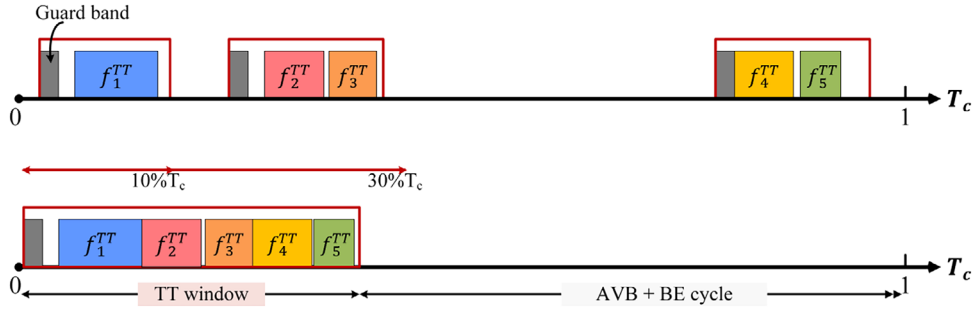


FIGURE 8 The time-triggered (TT) windows schematic diagram with porosity configuration and compact back-to-back configuration for TT frames.

where $L_{GB} = \max\{l_{AVB}^{max}, l_{BE}^{max}\}$ is the maximum frame size compatible with inaccuracy in the worst-case. % is the remainder operation, and it has the same meaning in the later formula. $\theta_{\lfloor \frac{\Phi[k,e]}{T_c} \rfloor}$ is the percentage of time cycle occupied by TT windows in the cycle $\lfloor \frac{\Phi[k,e]}{T_c} \rfloor$. More specific scheduling constraints based on no waiting scheduling are given uniformly in the following and the detailed descriptions are referred to [30]. T_c takes the value of the CQF time slot corresponding to AVB-A traffic.

4.2.2 | Transmission constraints

These constraints ensure that the constraints on scheduling decision variables work when link e is occupied by flow f_k^{TT} .

$$\Phi[k, e] \leq M_1 \times R[k, e], \forall f_k^{TT} \in F^{TT}, \forall e \in E, \quad (3)$$

where M_1 is an arbitrary but sufficiently large constant which is introduced to ensure that the constraint only work when the link e is occupied by the traffic f_k^{TT} . $R[k, e]$ is the route allocation.

4.2.3 | Contention constraints

These constraints require that all assigned time slots are non-overlapping to achieve zero congestion transmission.

$$\begin{aligned} & \forall f_m^{TT}, f_k^{TT} \in F^{TT} (m \neq k), \forall e \in E, R[m, e] + R[k, e] \geq 2 : \\ & \Phi[m, e] + \omega[m, e] \times f_m^{TT}.prd \geq \Phi[k, e] + \omega[k, e] \times f_k^{TT}.prd + f_k^{TT}.td, \\ & \text{or} \\ & \Phi[k, e] + \omega[k, e] \times f_k^{TT}.prd \geq \Phi[m, e] + \omega[m, e] \times f_m^{TT}.prd + f_m^{TT}.td. \end{aligned} \quad (4)$$

where $\omega[k, e] \in [0, \frac{T_{GCL}}{f_k.prd} - 1]$ is the transmission window index which is the positive integer multiple of the flow period $f_k^{TT}.prd$. $f_k^{TT}.td$ and $f_m^{TT}.td$ is the transmission duration of f_k^{TT} and f_m^{TT} on the link, respectively, which can be derived from dividing the data frame length by the link bandwidth.

4.2.4 | Transmission cycle constraints

The flow must be transmitted during its cycle and cannot be confused with the next cycle.

$$\forall f_k^{TT} \in F^{TT}, \forall e \in R_k : 0 \leq \Phi[k, e] \leq f_k^{TT}.prd - f_k^{TT}.td, \quad (5)$$

$$\forall f_k^{TT} \in F^{TT}, \forall e \in R_k : \Phi[k, e] + \omega[k, e] \leq M_2 \times R[k, e] \quad (6)$$

where M_2 is an arbitrary but sufficiently large constant.

4.2.5 | Path order constraints

These constraints require that the previous node can start to transmit the next frame when the frame is received by the current node.

$$\forall f_k^{TT} \in F^{TT}, \forall e_1 = (v_x, v_y), e_2 = (v_y, v_z) \in R_k : \quad (7)$$

$$\Phi[k, e_2] \geq \Phi[k, e_1] + f_k^{TT}.D - M_3 \times (1 - R[k, e_2])$$

where M_3 is an arbitrary but sufficiently large constant. $f_k^{TT}.D$ is the delay for each hop, which consists of transmission duration, propagation delay, and processing delay. This inequality works only when the link e was occupied by the traffic f_k^{TT} .

4.2.6 | Latency constraints

The maximum end-to-end latency constraint should be satisfied.

$$\forall e_1 \in R_k, e_1[0] = f_k^{TT}.src, \forall e_n \in R_k, e_n[1] = f_k^{TT}.dest :$$

$$\begin{aligned} f_k^{TT}.L & \triangleq (\Phi[k, e_n] + \omega[k, e_n] \times f_k.prd) \\ & - (\Phi[k, e_1] + \omega[k, e_1] \times f_k.prd), \end{aligned}$$

$$f_k^{TT}.L \leq f_k.ddl - f_k.td. \quad (8)$$

where $f_k^{TT}.L$ is the time difference between sending from the last switching node and sending from the first switching node. R_k is the route in the form of ordered links.

4.2.7 | No wait constraints

These constraints assume that each node must forward the frame immediately once it is received.

$$\forall f_k^{TT} \in F, \forall e_1 = (v_x, v_y), e_2 = (v_y, v_z) \in R_k : \quad (9)$$

$$\Phi[k, e_2] + \omega[k, e_2] \times f_k^{TT} \cdot prd = \Phi[k, e_1] + \omega[k, e_1] \times f_k^{TT} \cdot prd + f_k^{TT} \cdot td$$

4.2.8 | Objective function 1

The optimization goal of the TT scheduling problem is to minimize the end-to-end latency and the sending start time of each flow.

$$schedule(\Phi) = \sum_{f_k \in F} f_k \cdot L + \Phi[k, R_k[0]] \quad (10)$$

The CQF shaper divides the time slot into a series of equal intervals, each of which is called a cycle T_c , which should be set large enough to accommodate the traffic transmissions that occur during the class measurement interval (CMI) and one maximum interfering frame. Delaying the per-flow sending time on end systems can improve the queue resources utilization [12]. We modify the original CQF mechanism to reserve an exclusive window for TT traffic in each cycle. Different gate control situations for queues are presented in Figure 9. The core idea is to control the gate based on exclusive time slot slicing for each TT queue, where the AVB queue is closed when the TT queue is opened (including the receiving queue and the sending queue). When the TT queue gate is closed, the AVB queue is opened and closed by the setting of the cycle. All subsystems are non-preemptive. Since the scheduling of TT flows is optimized on a flow-by-flow level, the transmission planning results for TT flows are known for subsequent scheduling. The important input of the scheduling algorithm is the cycle, that is, T_c^{class} , $class \in \{A, B\}$, and the CQF queue size. Since any change of T_c^{class} will need to restart the entire system, it is necessary to set a proper value at the initial state of the system. A different group of CQF queues can consider different cycles. We define the minimal cycle as T_c . The cycles T_c^A and T_c^B are positive integer multiple of T_c . Without loss of generality, we assume that the starting of each cycle at different nodes is the same, which can ensure that all CQF queues will empty before the end of their cycle. The detailed analysis and proof can be directed to [31].

4.2.9 | Time slot constraints

The lower and upper bounds of the time slot size T_c^{class} , $class \in \{A, B\}$ are given in this constraint. For the upper bounds of T_c^{class} , it is greater than the greatest common divisor (GCD) of flow periods, while for the lower bounds of T_c^{class} , the largest

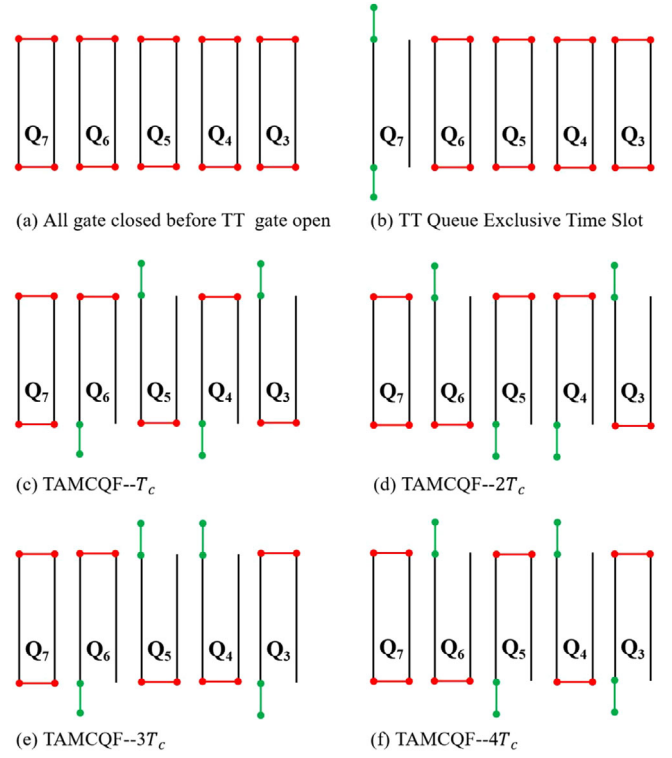


FIGURE 9 All possible combinations of gate state situations for time-triggered (TT) queue and two audio video bridging (AVB) queues with different cycles in the designed hybrid scheduling ($T_c^A = T_c$, $T_c^B = 2T_c$).

packet among AVB flows should be accommodated. Besides, T_{GCL} should be divisible by T_c^{class} . When the cycle and path are determined, $f_k^{AVB} \cdot WCD$ is known by the number of hops and T_c^{class} , which is not affected by interference from other unscheduled traffic.

$$\begin{aligned} C1 : T_c^{class} &\leq \text{GCD}(F^{AVB} \cdot \text{periods}), \\ C2 : T_c^{class} &\geq \frac{L_{queue}}{e \cdot R} + t_{hop_delay} + t_{sync}, \\ C3 : T_{GCL} \% T_c^{class} &= 0, \\ C4 : T_c^{class} &= M \cdot T_c, \forall M \in \mathbb{Z}^+. \end{aligned} \quad (11)$$

where $e \cdot R$ denotes the link bandwidth, L_{queue} is the maximum amount of bytes that each queue can hold, and t_{hop_delay} , t_{sync} represent the frame delay in one hop and the clock synchronization, respectively.

4.2.10 | Transmission cycle constraints

These constraints require that the sending time of the frame should be within its period. It helps reduce the search space since the limited range of the offset is reasonable.

$$\forall f_k^{AVB} \in F^{AVB} : 0 \leq f_k^{AVB} \cdot \phi < \frac{f_k^{AVB} \cdot prd}{T_c^{class}} \quad (12)$$

4.2.11 | Deadline constraints

These constraints restrict all traffic arriving at the destination before the deadline.

$$\forall f_k^{AVB} \in F^{AVB} : \quad f_k^{AVB} \cdot \phi + \text{len}(f_k^{AVB} \cdot \text{route}) - 1 \leq \frac{f_k^{AVB} \cdot \text{ddl}}{T_c^{\text{class}}} \quad (13)$$

where $\text{len}(f_k^{AVB} \cdot \text{route})$ is the length of route reserved for the flow f_k^{AVB} .

4.2.12 | Queue resource constraints

A resource block $B[\text{link}, \text{slot}]$ is the queue and time resource on one port of a switch. If the flow occupies a resource block, $\text{Map}(f_k^{AVB}, o[k, e]) = 1$, and each resource block can be occupied by multiple traffic simultaneously, which can effectively improve the scheduling success rate and bandwidth utilization. The usage of each queue resource block should not exceed the queue length as well as the reserved transmission capacity defined in constraints 14 and 15.

$$\begin{aligned} \forall e \in f_k^{AVB} \cdot \text{route}, \\ \forall \rho[k] \in \left[0, \frac{T_{GCL}}{f_k^{AVB} \cdot \text{prd}}\right), \forall \text{slot}[e] \in \left[0, \frac{T_{GCL}}{T_c^{\text{class}}}\right) : \\ o[k, e] = \left(f_k^{AVB} \cdot \phi + \frac{\rho[k] \times f_k^{AVB} \cdot \text{prd}}{T_c^{\text{class}}} + \text{index}(SW) \right) \% \left(\frac{T_{GCL}}{T_c^{\text{class}}} \right), \\ \text{Map}(f_k^{AVB}, o[k, e]) = \begin{cases} 1, \forall o[k, e] \in o[k, e], \\ 0, \forall o[k, e] \in \text{slot}[e] \setminus o[k, e], \end{cases} \\ \sum_{k=0}^{K_{AVB}} f_k^{AVB} \cdot \text{size} \times \text{Map}(f_k^{AVB}, o[k, e]) \leq L_{\text{queue}}, \end{aligned} \quad (14)$$

where $\rho[k]$ is index to the period instances of f_k^{AVB} in T_{GCL} . And $o[k, e]$ is the index to the cycle instances of f_k^{AVB} on the link e .

4.2.13 | Bandwidth resource constraints

These constraints restrict that the data rate of each priority traffic does not exceed the reserved bandwidth, and the total data rate of each port cannot exceed the link bandwidth.

$$\begin{aligned} C1 : \frac{\sum_{k=0}^{K_{AVB}} f_k^{AVB} \cdot \text{size} \times \text{Map}(f_k^{AVB}, o[k, e])}{(1 - \theta_{o[k, e]}) \times T_c^{\text{class}}} &\leq R_{\text{class}}, \\ C2 : \sum_P R_{\text{class}} &\leq BW \end{aligned} \quad (15)$$

where R_{class} is the data rate of AVB-class traffic.

ALGORITHM 1 Proposed TSO

Input: Network topology G , Flow set F , T_c^A, T_c^B
Output: $\Phi[f^{\text{tt}}][e], \phi[f^{\text{avb}}]$, unscheduled flow set U

- 1 Initialize all queue resource $R[e, t]$
- 2 $F^{\text{tt}}, F^{\text{avb}} = \text{flow_classification}(F)$
- 3 $R[f_k^{\text{tt}}] = \text{find_path}(f_k^{\text{tt}})$
- 4 $\Xi \leftarrow$ time-sensitive constraints for TT traffic
- 5 $\text{milp} \leftarrow \text{gurobi.model}()$
- 6 $\text{milp.add_constraints}(\Xi)$
- 7 $\Phi, \text{done} \leftarrow \text{solveMILP}()$
- 8 **for each** $f_k^{\text{avb}} \in F^{\text{avb}}$ **do**
- 9 $R[f_k^{\text{avb}}] = \text{find_path}(f_k^{\text{avb}})$
- 10 **for** $o\text{ffset} = \text{Selection}(f_k^{\text{avb}} \cdot \text{prd})$ **do**
- 11 $e2e = \text{calculate_e2e}(o\text{ffset})$
- 12 **if** $e2e \leq f_k^{\text{avb}} \cdot \text{ddl}$ **&**
- 13 $\text{check_queue_resource}(f_k^{\text{avb}})$ **&**
- 14 $\text{check_bandwidth_resource}(f_k^{\text{avb}} \cdot P)$ **then**
- 15 $\phi[f_k^{\text{avb}}] = o\text{ffset}$
- 16 $\text{update_queue_resource}(e, t)$
- 17 $\text{update_bandwidth_resource}(f_k^{\text{avb}} \cdot P)$
- 18 **end**
- 19 **end**
- 20 **end**

4.2.14 | Objective function 2

$\min L(\Phi_{AVB})$ aims to map more flows in the network and load balance the time slot resource blocks, which can be achieved by min-max allocation in a natural way. We try to find the minimization of the bandwidth occupancy in the busiest time slot of the link. And $L(\Phi_{AVB})$ is expressed as:

$$\max_o \sum_{k=0}^{|F_{AVB}|-1} \frac{f_k^{AVB} \cdot \text{size}}{L_{\text{queue}}} \times \text{Map}(f_k^{AVB}, o[k, e]) \quad (16)$$

5 | HYBRID TRAFFIC SCHEDULING SOLUTION

In this section, we propose a hybrid traffic scheduling solution to find optimal results within the TSN network. The traffic scheduling involves the configuration of GCL in each output. These decisions are subject to all the scheduling constraints for different priority class traffic fulfilment.

The procedures of TSO are presented in Algorithm 1 as follows. The inputs of TSO are the Network topology G , Flow set F , and time cycle of each priority class T_c^A, T_c^B . When a valid schedule is found, the output $\Phi[f^{\text{tt}}][e], \phi[f^{\text{avb}}]$ indicates the flow allocation of each frame. $\Phi[f^{\text{tt}}][e]$ implies the

starting position of the time slot for the TT frames on each egress port, and $\phi[f^{avb}]$ signifies the index of the cycle in the source of AVB frames. In the initialization step, all queue and time slot resources are initialized. The flow classification based on priority identification is invoked (line 2) to obtain $F^{\#}$ and F^{avb} . Here, we assume that the paths of all traffic are known in advance (line 3 and line 9). The corresponding MILP constraints detailed in the above section are generated for $F^{\#}$. The function *solveMILP()* searches for a feasible optimal solution that satisfies these constraints. The Boolean variable *done* is *True* when a feasible solution is found. Other cases (i.e. timeout, infeasible) will return *False*. Then the incremental scheduling solution is performed for F^{avb} (lines 8 to 19). The local search adapts the greedy heuristics to pick the injection time with the minimum occupied bandwidth combination via path. We select the optimal solution under the current scheduling results in order to reduce execution time. If all the constraints are satisfied, the injection time on its source is set as the value *offset*. Then the resource pool is updated since the traffic has reserved the corresponding queue time slot resource along the path. If the solution cannot be found after iterating through all possible time slots under the previous scheduling result, then the flow is placed into the set unscheduled flow set **U**. The worst-case time complexity of the proposed TSO algorithm is $O((|F^{\#}| \times |E|)^m + |F^{avb}| \times N)$, where $|F^{\#}| \times |E|$ and m are the number of variables and constraints in ILP problem, respectively. $|F^{avb}|$ is the number of AVB traffic and N is the number of flow injection time slots. The proposed TSO algorithm consists of two main components, TT traffic scheduling and AVB traffic scheduling. The scheduling of TT traffic is based on the linear integer programming optimization to find the global optimal solution, which is NP-hard with exponential time solvable complexity. Its time complexity is related to the number of variables and constraints. The scheduling of AVB traffic contains two layers of nested loops that traverse each AVB traffic and the optional time slot respectively.

6 | SIMULATION RESULTS

6.1 | Hybrid traffic scheduling model evaluation

6.1.1 | Evaluation setup

In this section, we evaluate the hybrid traffic scheduling model in a local real automotive network using OMNeT++ [36] environment based on the simulation model CoRE4INET [37], which extends the INET Framework. All experiments and evaluations run on a PC equipped with 64bit 2.1GHz Intel(R) Core(TM) i7-12700F CPUs and 32 GB RAM.

Figure 10 depicts the topology of the simulation, where there are five end systems and two TSN switches and all devices are connected to each other by a 1 m-long Ethernet link with bandwidth 1Gbps. This scenario can simply represent hybrid

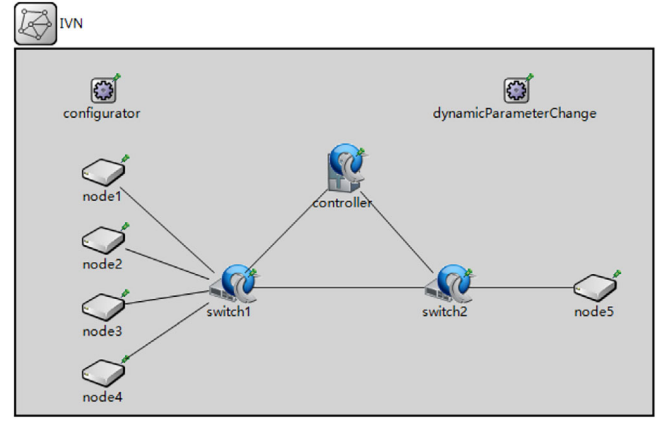


FIGURE 10 The simulation topology.

TABLE 4 Flow configuration in the simulation.

No.	Priority class	Size	prd	src	Destination
1	TT	100 bytes	1000 μ s	Node 1	Node 5
2	AVB-A	1500 bytes	125 μ s	Node 2	Node 5
3	AVB-B	1500 bytes	250 μ s	Node 3	Node 5
4	BE	100–1500 bytes	random	Node 4	Node 5

traffic transmission cases in TSN. Node 5 is the general command which collects information from nodes 1 to 4. Node 1 is a control actuator to return feedback messages to node 5 to collaborate with multiple actuators. Node 2 and node 3 are sensors that transfer audio and video information to update environmental posture. Other normal information as best-effort flow is sent from node 4 to node 5. The related flow configuration is shown in TABLE 4. To present the scenario where traffic aggregation occurs in a queue, we increase the frame rate of the AVB flows. The GCL of TT flows is the same as that in TAS+CBS, TAS+CQF, and TAMCQF. The T_{GCL} in TAS+CBS and TAMCQF is 125 and 1000 μ s, respectively, and others are 250 μ s. The gate of BE flow is only closed when the TT gate is open in all mechanisms.

In the CQF mechanism, the gates of TT and BE flows are always kept open. The AVB flows are mapped to the Q_5 and Q_6 alternately under the CQF and TAS+CQF mechanism. The AVB-A flows are mapped to the Q_6 and Q_5 queues and the AVB-B flows are mapped to the Q_4 and Q_3 queues under the TAMCQF mechanism. To increase the number of flows, we generate a BE flow every 1000 μ s randomly. The cycle T_c is 125 μ s under the CQF and TAS+CQF mechanism, T_c^A and T_c^B are 125 and 250 μ s, respectively, under the TAMCQF model. We modified the structure of the port in the.ned file, as shown in Figure 11. It adds a set of gates after the incoming queue, and only the queue using the multi-CQF model needs to update the incoming gates by gate control list, the rest of the incoming gates are always in the open state.

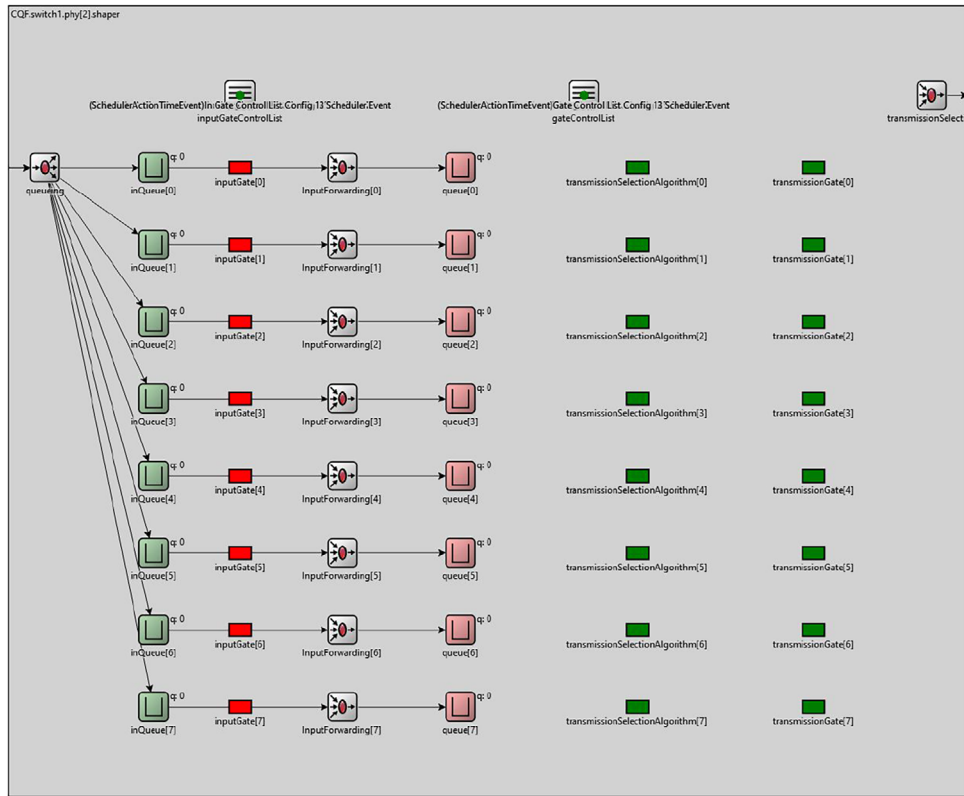


FIGURE 11 TSNPHYPort architecture with time-aware multi-cyclic queuing and forwarding (TAMCQF) scheduling model in time-sensitive networking (TSN) switch.

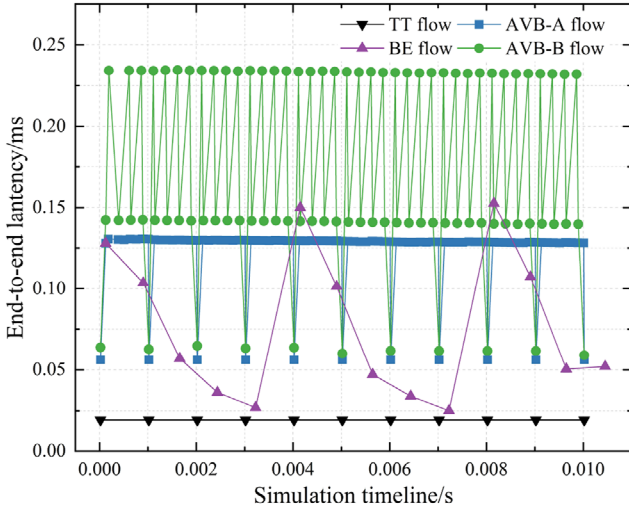
6.1.2 | End-to-end latency versus scheduling model

The simulation results for four scheduling models are presented from Figures 12 to 15. All scheduling models can ensure the bounded latency of TT flows and AVB flows. In these diagrams for the simulation characterizing the end-to-end latency, the y -axis indicates the end-to-end latency from the talkers to the listener, whereas the x -axis indicates the timeline in the simulation. Figure 12 shows the simulation results of the switch with the combined CBS and TAS model. In the simulation setup, ten percent of the bandwidth is reserved for TT flows and 75% for AVB flows, and BE flows use the remaining bandwidth. As expected, AVB flows and BE flows have no interference with the transmission of TT flows. TT flows can achieve zero-jitter transmission, while AVB flows show higher jitter than TT flows. AVB flows can be delivered within a bounded delay, while BE flows under strict priority-based scheduling are not guaranteed. As shown in Figure 12b, the figure compares the latency distribution of different classes of traffic under the combined CBS and TAS model, where the horizontal axis indicates the traffic class and the vertical axis indicates the end-to-end latency. In each traffic class, the box plot is shown on the left, and the point scatter plot is shown on the right. From the mean value of latency, $TT < BE < AVB-A < AVB-B$. As for the degree of scattered distribution, $TT < AVB-A < AVB-B < BE$, in which the latency of AVB-A flow is mainly centralized

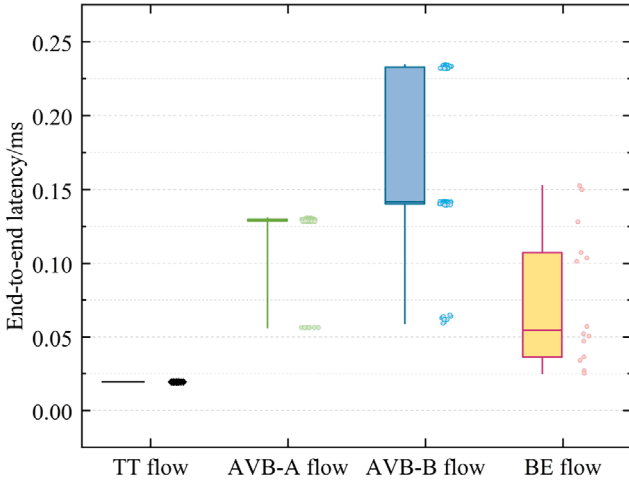
around 1.29×10^{-4} s, whereas the latency of AVB-B flow presents two concentrations, respectively, around 2.34×10^{-4} and 1.42×10^{-4} s. In contrast, the BE flows show a lower mean value of latency, but the distribution of latency is more fragmented. Of note is that the AVB class flows all show a minimum value of latency after the TT window is closed.

Figure 13 presents the simulation results for end-to-end latency of AVB flows with CQF model and TAS+CQF model. The end-to-end delay of AVB class traffic under the above two models is distributed around with 2.6×10^{-4} s. Since the same single cycle is adapted to the queue, there is no difference in the boundary of latency for AVB-A flows and AVB-B flows. The range of end-to-end latency is associated with the cycle and the number of hops. Since the CQF scheduling model dispatches TT flows together with other flows, the end-to-end latency of TT flows is affected to some extent by the blocking of AVB flows and BE flows. Nevertheless, a strict priority scheduling rule is adhered to, whereby TT traffic is prioritized and scheduled with lower latency than AVB class traffic. The Simulation results for end-to-end latency of TT flows are shown in Figure 14. TT flows are scheduled based on exclusive windows in the TAS+CQF scheduling model, which outperforms the CQF scheduling model in terms of latency and jitter.

The simulation results of the TAMCQF model are shown in Figure 15. The Multi-CQF queue model utilizes different cycles so that the latencies of AVB flows are within different boundaries, all of which satisfy the requirements for end-to-end



(a) End-to-end latency comparison over timeline.



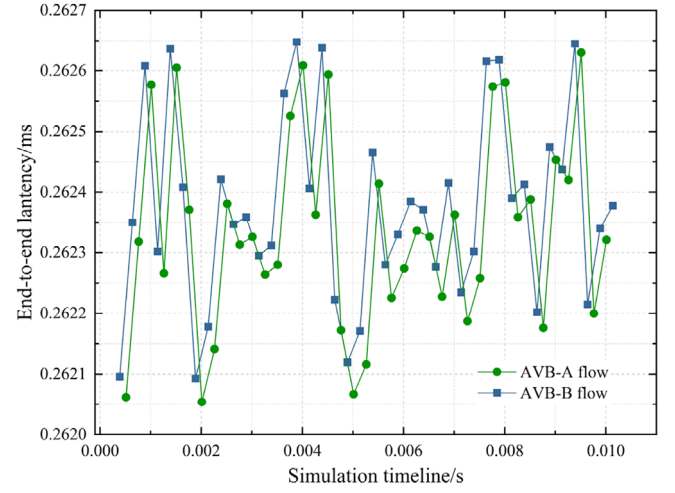
(b) Latency distribution comparison.

FIGURE 12 Simulation results for end-to-end latency of hybrid traffic with time-aware shaper (TAS)+credit-based shaper (CBS) model.

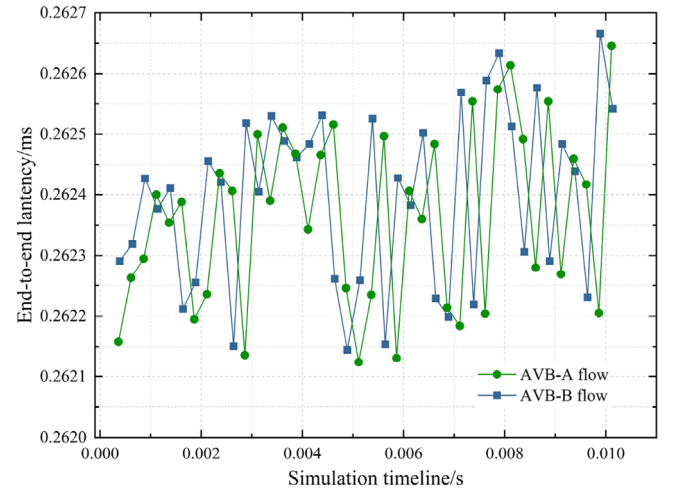
latency. As expected, TT flows can be dispatched with zero jitter and low latency. Although the TAMCQF scheduling model increases the latency of AVB-B flows to some extent, the search space will be greatly reduced during the scheduling process, which is qualified with the scheduling capability metric in the next subsection. We statistically measure the end-to-end latency metrics of BE flows within 0.058s, as shown in Figure 16. In terms of latency mean and jitter, the use of the CQF-type queue scheduling model has similar results, enabling more full utilization of the remaining bandwidth compared with TAS+CBS model without affecting the performance of other classes of traffic.

6.1.3 | Schedulability capacity versus scheduling model

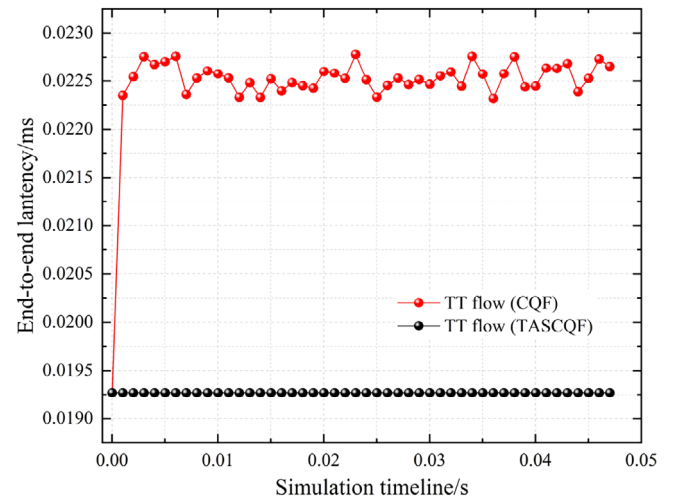
We generate the test cases for the MILP instances in a python-based framework, which were solved in Gurobi optimizer



(a) CQF model.



(b) TAS+CQF model.

FIGURE 13 Simulation results for end-to-end latency of audio video bridging (AVB) flows with cyclic queuing and forwarding (CQF) model and time-aware shaper (TAS)+CQF model.**FIGURE 14** Simulation results for end-to-end latency of time-triggered (TT) flows with cyclic queuing and forwarding (CQF) model and time-aware shaper (TAS)+CQF model.

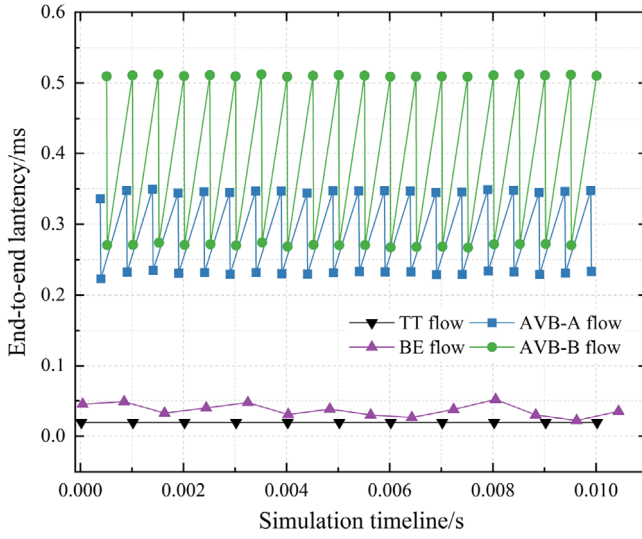


FIGURE 15 Simulation results for end-to-end latency of hybrid traffic with time-aware multi-cyclic queuing and forwarding (TAMCQF) model.

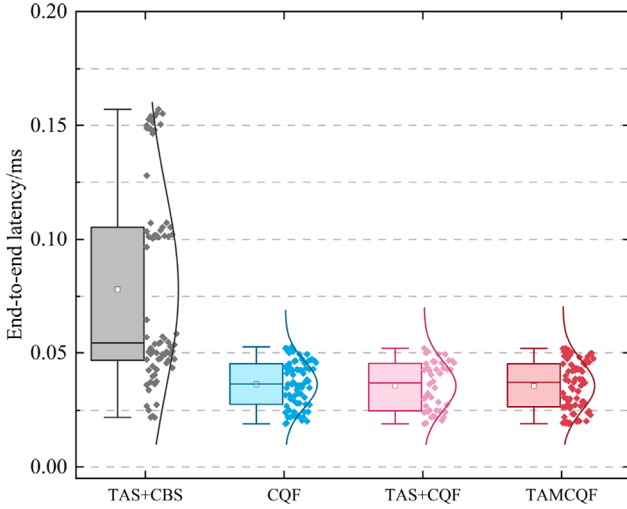


FIGURE 16 Simulation results for end-to-end latency distribution of best-effort (BE) flows with different scheduling model.

(version 9.5.1, 64-bit Windows) [38]. We create network topologies by NetworkX graph library and generate different traffic patterns. In this subsection, we conduct experiments on a ring topology, which is a typical topology in an automotive network. To evaluate the scalability of the proposed scheduling model, we use two network scales, which consist of 4 and 16 switches, respectively. The link bandwidth is set to 1Gbps, $T_c = 125 \mu s$, $T_c^A = 125 \mu s$, and $T_c^B = 250 \mu s$. The queue capacity of class A and class B is set to 10 MTU and 20 MTU Ethernet frames respectively. In the CQF scheduling model, $T_c = 125 \mu s$ and the CQF queue capacity is set to 10 MTU.

Each flow's source and destination end systems are randomly chosen from the ESs to cover the unicast and multicast broadcast. The TT traffic period is randomly generated from the set $\{500, 1000\} \mu s$ and the AVB traffic period is randomly generated from the set $\{4, 8\} ms$. The TT traffic deadline is equal to

TABLE 5 Percentage of solved, timeout, and infeasible schedules of 100 synthetic test cases using MILP technique with different scheduling models for different numbers of traffic.

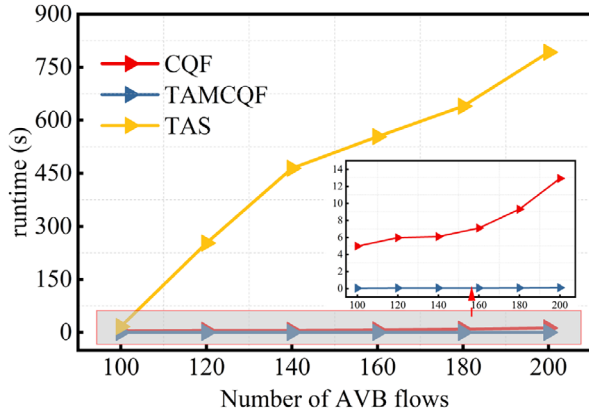
Scheduling model	#TT	#AVB	#Solved	#Timeout	#Infeasible
TAS	50	120	100	0	0
TAS	50	140	100	0	0
TAS	50	160	54	46	0
TAS	50	180	34	66	0
TAS	50	200	14	86	0
TAS+CQF	50	1000	100	0	0
TAS+CQF	50	1500	95	0	5
TAS+CQF	50	1700	41	40	19
TAS+CQF	50	1800	23	49	28
TAMCQF	50	1000	100	0	0
TAMCQF	50	1500	100	0	0
TAMCQF	50	1700	100	0	0
TAMCQF	50	1800	100	0	0

its period, and the AVB traffic deadline is randomly generated from $[\eta \cdot period, period]$. The default value of η is 0.4. The frame size of each TT traffic is randomly selected from the range $[100, 125]B$, and the frame size of each AVB traffic is randomly selected from the set $\{1, 1.2, 1.5\}KB$. For each parameter setting, we run 100 synthetic test cases.

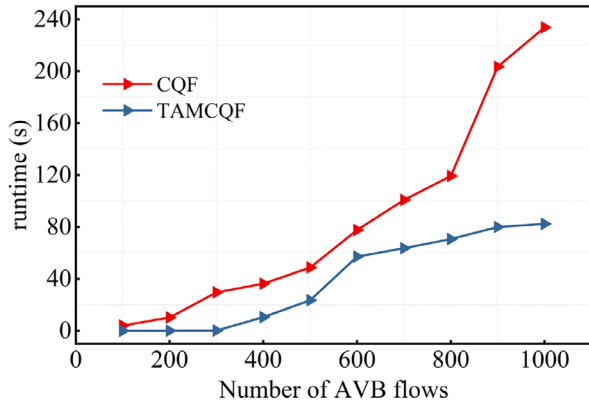
Each MILP model finishes with one of the following three possible states: a) optimal: one sub-optimal result found, b) infeasible: no schedule result was found with the given traffic, and c) timeout: the time limit was reached before the result was found. We generate 100 instances on each MILP model and conduct statistics on the scheduling results. The schedulability performance of different mechanisms is shown in Table 5. With the timeout of 1200 s, the MILP scheduling results of the TAS scheduling model can achieve all successful scheduling of 50 TT flows and 140 AVB flows. For more than 140 AVB flows, most instances will terminate the search due to timeout, while the CQF model and TAMCQF model can effectively improve bandwidth utilization. Especially, the TAMCQF model has a significant advantage in terms of optimally solved patterns. This is simply reasoned by the different scheduling granularity provided for different traffic.

6.1.4 | Execution time versus scheduling model

As shown in Figure 17, the CQF model cannot schedule TT flows when the network size is large, due to the latency range $[(len(route) - 1) \times T_c, (len(route) + 1) \times T_c]$ constraint. In a small network (Scenario 1), 50 TT flows and N AVB traffic traverse the networks. CQF and TAMCQF models effectively reduce runtime by increasing the scheduling granularity of the AVB traffic while the runtime of TAS increases sharply with the number of flows due to the strict constraint by microsecond level shown in Figure 17a. In Scenario 2, there is no TT traffic in the



(a) Scenario 1: Ring topology with 4SWs and 4ESs.



(b) Scenario 2: Ring topology with 16SWs and 10ESs.

FIGURE 17 Evaluation results of runtime with different scheduling model.

network. TAMCQF model uses different cycles to increase the scheduling granularity of AVB-B traffic, effectively reducing the search space shown in Figure 17b. TAS is almost infeasible in Scenario 2 since the size of the network is large and the search space is too huge.

6.2 | TSO performance evaluation

We carried out an extensive evaluation of the performance of TSO in the TAMCQF scheduling model. We compared TSO with the MILP optimal solution and the start-slot assignment (SSA) solution. MILP optimal solution invokes the ILP solver Gurobi to try to find the near-optimal schedule, and SSA is to find the earliest injection time flow-by-flow selected by order from the scheduled flow set.

6.3 | Evaluation setup

6.3.1 | Network and resource setting

We generate the problem instances for the experiments with Python, in which we can create topologies by using the

Networkx graph library. All experiments and evaluations run on a PC equipped with 64bit 2.1GHz Intel(R) Core (TM) i7-12700F CPUs and 32 GB RAM. we conduct experiments on the ring topologies with 16 switches, 10 end systems, and 29 links to evaluate our proposed scheduling solution based on our proposed scheduling model in this subsection.

6.3.2 | Traffic generation

For each flow, the source and destination are generated by picking from the ESs randomly with uniform probability. The period of each flow is milliseconds in general. We simulate $f^{th}.prd$ by randomly drawing one value from the set of $\{1, 2\}$ ms and $f^{AVB}.prd$ from the set of $\{4, 8\}$ ms. Each deadline is randomly generated from $[\eta \cdot period, period]$. The default value of η is 0.4. The size of each TT flow is selected from the range $[100, 125]B$, and the size of each AVB flow is selected from the set $\{1, 1.2, 1.5\}KB$.

6.3.3 | Evaluation for the impact on the execution time

As shown in Figure 18, the time overhead of proposed TSO scheme is exhibited and compared with classical SSA and MILP optimal solution. Note that the time overhead of the MILP optimal algorithm consists of model construction overhead, solver invocation overhead, and result processing overhead. The timer was called at the beginning of the algorithm execution until the scheduling results was obtained. The overall runtime of the algorithm was recorded. Figure 18a shows the runtime of the MILP optimal solution and Figure 18b shows the runtime of the SSA and proposed TSO solution. From the simulation results, all three scheduling solutions are related to the number of traffic, and the execution time of SSA and TSO algorithms can achieve the scheduling of 1000 traffic flows at the seconds level, while MILP optimal algorithm takes hundreds of seconds. The runtime of the SSA solution varies significantly with the increase of traffic, because the sequential use of time slot resources causes some resource blocks to be full and cannot be used by other traffic, and the number of iterations increases when the number of traffic becomes larger, so a faster response than the TSO solution can be achieved before 400 streams, while the runtime after 400 traffic all exceeds that of TSO solution. The average runtime of all scheduling cases for AVB traffic from 100 to 1000 is shown in Figure 18c, and as a whole, the average runtime of TSO algorithm is much smaller than the MILP optimal algorithm.

6.3.4 | Evaluation for the impact on the scheduling capability

The scheduling capability is presented as the average number of scheduled traffic. The number of AVB traffic is divided into four levels from 2500 to 4000 to present the performance of

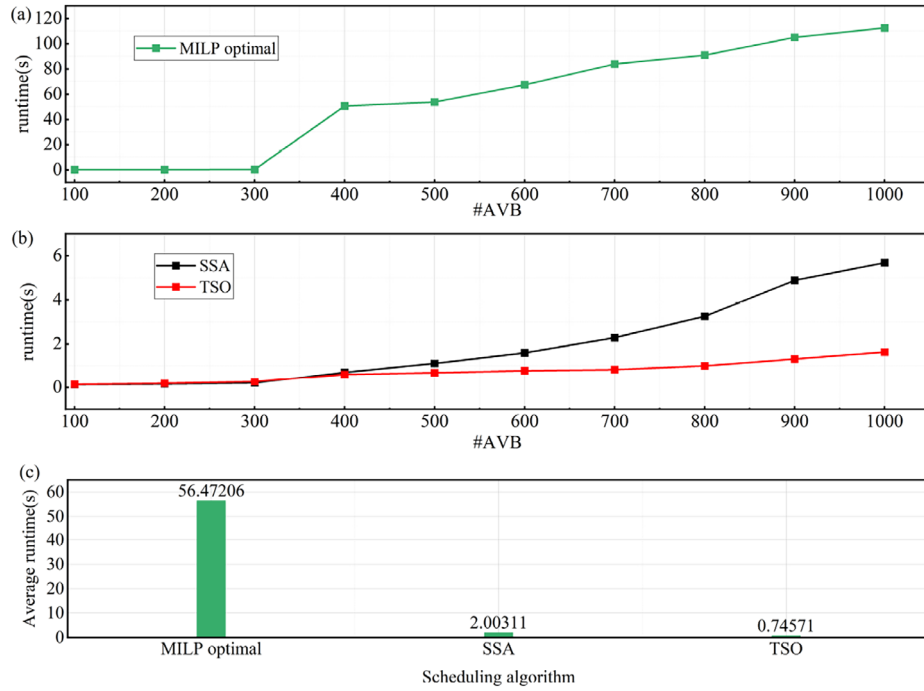


FIGURE 18 Evaluation results (runtime) for the impact on the number of scheduled traffic of scheduling algorithm based on time-aware multi-cyclic queuing and forwarding (TAMCQF) model with 100 time-triggered (TT) traffic and N audio video bridging (AVB) traffic.

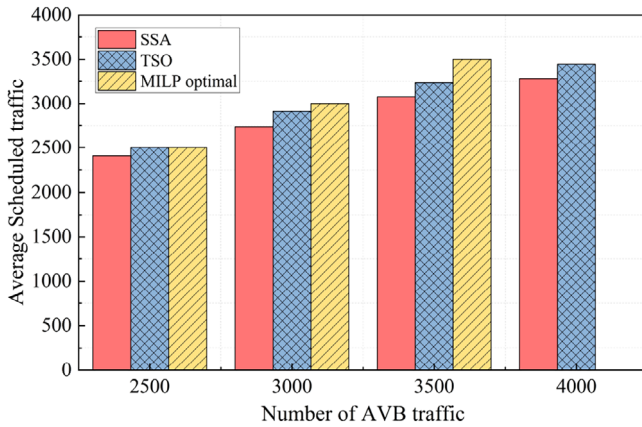


FIGURE 19 Evaluation results (scheduling capability) for the impact on the number of scheduled traffic of time-aware multi-cyclic queuing and forwarding (TAMCQF) scheduling model with 20 time-triggered (TT) traffic and N audio video bridging (AVB) traffic.

schedulability as the number of traffic increases. As shown in Figure 19, the scheduling rate of the SSA method is lower than the other two methods in different groups of experiments. And TSO solution can achieve a scheduling capability similar to MILP optimal solution, and the MILP solver is almost impossible to solve when the traffic exceeds 3500.

7 | CONCLUSIONS

In this article, a novel hybrid traffic scheduling model TAMCQF was proposed and discussed for TSN, satisfying QoS

requirements for various applications. Then, we convert TAMCQF into a mathematical model and formally define and form a MILP problem. The proposed TSO solution was considered to find the optimal schedule based on the TAMCQF model. Based on the OMNeT++ framework, we developed the simulation of the scheduling model. The simulation results in the evaluation scenario show that the TAMCQF scheduling model can guarantee zero jitters for TT flows and a bounded delay for AVB flows. Three experiments were conducted using 100 test cases to evaluate the scalability of three scheduling models, which showed the out-performance of the TAMCQF model in terms of runtime and scheduling capability. Additionally, we carried out an extensive evaluation of the performance of TSO in the TAMCQF scheduling model. The results demonstrated that the proposed scheduling solution considerably increases the number of TSN flows that can be satisfactorily served in the TSN network compared to common scheduling solutions. Besides, TSO can obtain near-optimal solutions that balance the solution quality in terms of runtime. In the future, we will consider experiments oriented to real-world scenarios to solve the scheduling problems in IVN.

AUTHOR CONTRIBUTIONS

Hongrui Nie: Writing—review and editing. **Yue Su:** Review and editing. **Weibo Zhao:** Writing—review and editing. **Junsheng Mu:** Writing—review and editing.

CONFLICT OF INTEREST STATEMENT

The authors declare no conflicts of interest.

DATA AVAILABILITY STATEMENT

Data available on request from the authors.

ORCID

Hongrui Nie  <https://orcid.org/0000-0002-3833-4490>

REFERENCES

- Bello, L.L., Mariani, R., Mubeen, S., Saponara, S.: Recent advances and trends in on-board embedded and networked automotive systems. *IEEE Trans. Ind. Inf.* 15(2), 1038–1051 (2018)
- Nayak, N.G., Dürr, F., Rothermel, K.: Incremental flow scheduling and routing in time-sensitive software-defined networks. *IEEE Trans. Ind. Information.* 14(5), 2066–2075 (2018)
- IEEE: 802.1Q-IEEE Standard for Local and Metropolitan Area Networks—Bridges and Bridged Networks. IEEE, Piscataway, NJ (2018)
- IEEE: IEEE Standard for Local and Metropolitan Area Networks – Frame Replication and Elimination for Reliability. IEEE, Piscataway, NJ (2017)
- Gavrilut, V., Pruski, A., Berger, M.S.: Constructive or optimized: An overview of strategies to design networks for time-critical applications. *ACM Comput. Surv.* 55(3), 1–35 (2022)
- Xu, L., et al.: Learning-based scalable scheduling and routing co-design with stream similarity partitioning for time-sensitive networking. *IEEE Internet Things J.* 9(15), 13353–13363 (2022)
- Bush, S.F.: Toward efficient time-sensitive network scheduling. *IEEE Trans. Aerosp. Electron. Syst.* 58(3), 1830–1842 (2022)
- Feng, Z., Cai, M., Deng, Q.: An efficient pro-active fault-tolerance scheduling of IEEE 802.1Qbv time-sensitive network. *IEEE Internet Things J.* 9(16), 14501–14510 (2022)
- IEEE: IEEE Standard for Local and Metropolitan Area Networks—Audio Video Bridging (AVB) Systems. IEEE Std 802.1BA, pp. 1–45. IEEE, Piscataway, NJ (2011)
- Li, E., He, F., Li, Q., Xiong, H.: Bandwidth allocation of stream-reservation traffic in TSN. *IEEE Trans. Netw. Serv. Manage.* 19(1), 741–755 (2022)
- IEEE: Ieee/iso/iec international standard - information technology - telecommunications and information exchange between systems - local and metropolitan area networks - specific requirements - part 1q: Bridges and bridged networks - amendment 7: Cyclic queuing and forwarding. ISO/IEC/IEEE 8802-1Q:2016/Amd.7:2019(E), pp. 1–34. IEEE, Piscataway, NJ (2019)
- Yan, J., Quan, W., Jiang, X., Sun, Z.: Injection time planning: Making CQF practical in time-sensitive networking. *IEEE INFOCOM 2020—IEEE Conference on Computer Communications*, pp. 616–625. IEEE, Piscataway, NJ (2020)
- Zhang, Y., Xu, Q., Xu, L., Chen, C., Guan, X.: Efficient flow scheduling for industrial time-sensitive networking: A divisibility theory-based method. *IEEE Trans. Ind. Inf.* 18(12), 9312–9323 (2022)
- Nieuwenhuis, R., Oliveras, A., Tinelli, C.: Solving SAT and SAT modulo theories. *ACM, J.* 53(6), 937–977 (2006)
- Craciunas, S.S., Oliver, R.S., Chmelik, M., Steiner, W.: Scheduling real-time communication in IEEE 802.1Qbv time sensitive networks. In: *Proceedings of the 24th International Conference on Real-Time Networks and Systems*, pp. 183–192. Association for Computing Machinery, New York, NY (2016)
- Craciunas, S.S., Oliver, R.S.: An overview of scheduling mechanisms for time sensitive networks. In: *Proceedings of Real-Time Summer School L'École d'Été Temps Réel (ETR)*, pp. 1–7. TTTech Computertechnik AG, Vienna, Austria (2017)
- Pop, P., Raagaard, M.L., Craciunas, S.S., Steiner, W.: Design optimization of cyber-physical distributed systems using IEEE time-sensitive networks. *IET Cyber-Phys. Syst. Theory Appl.* 1(1), 86–94 (2016)
- Nayak, N.G., Dürr, F., Rothermel, K.: Routing algorithms for IEEE802.1Qbv networks. *ACM SIGBED Rev.* 15(3), 13–18 (2018)
- Wang, N., Yu, Q., Wan, H., Song, X., Zhao, X.: Adaptive scheduling for multicenter time-triggered train communication networks. *IEEE Trans. Ind. Informat.* 15(2), 1120–1130 (2018)
- Atallah, A.A., Hamad, G.B., Mohamed, O.A.: Routing and scheduling of time-triggered traffic in time-sensitive networks. *IEEE Trans. Ind. Inf.* 16(7), 4525–4534 (2020)
- Huang, Z., Zhu, H., Zhang, H., Huang, T.: A scalable heuristic time-sensitive traffic scheduling algorithm for in-vehicle network. In: *5th International Conference on Hot Information-Centric Networking (HotICN) 2022*, pp. 111–118. Guangzhou, China (2022)
- Bhattacharjee, S., Alexandris, K., Hansen, E., Pop, P., Bauschert, T.: Latency-Aware Function Placement, Routing, Scheduling in TSN-based Industrial Networks. *ICC 2022 - IEEE International Conference on Communications*, pp. 4248–4254. Seoul, Korea, Republic of (2022)
- Li, Y., Jiang, J., Hong, S.H.: Joint Traffic Routing and Scheduling Algorithm Eliminating the Nondeterministic Interruption for TSN Networks Used in IIoT. *IEEE Internet Things J.* 9(19), 18663–18680 (2022)
- Li, C., Zhang, C., Zheng, W., Wen, X., Lu, Z., Zhao, J.: Joint routing and scheduling for dynamic applications in multicast time-sensitive networks. In: *Proceedings of 2021 IEEE International Conference on Communications Workshops (ICC Workshops)*, pp. 1–6. IEEE, Piscataway, NJ (2021)
- Schweissguth, E., Timmermann, D., Parzyjegl, H., Danielis, P., Mühl, G.: ILP-based routing and scheduling of multicast realtime traffic in time-sensitive networks. In: *Proceedings of 2020 IEEE 26th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, pp. 1–11. IEEE, Piscataway, NJ (2020)
- Krolkowski, J., Martin, S., Medagliani, P., Leguay, J., Chen, S., Chang, X., et al.: Joint routing and scheduling for largescale deterministic IP networks. *Comput. Commun.* 165, 33–42 (2021)
- Yan, J., Quan, W., Jiang, X., Sun, Z.: Injection time planning: Making CQF practical in time-sensitive networking. In: *Proceedings of IEEE INFOCOM 2020—IEEE Conference on Computer Communications (INFOCOM)*, pp. 616–625. IEEE, Piscataway, NJ (2020)
- Zhao, L., Pop, P., Steinhorst, S.: Quantitative performance comparison of various traffic shapers in time-sensitive networking. *IEEE Trans. Netw. Serv. Manage.* 19(3), 2899–2928 (2022)
- Dürr, F., Nayak, N.G.: No-wait packet scheduling for IEEE time-sensitive networks (TSN). In: *Proceedings of the 24th International Conference on Real-Time Networks and Systems*, pp. 203–212. Association for Computing Machinery, New York, NY (2016)
- Atallah, A.A., Hamad, G.B., Mohamed, O.A.: Routing and scheduling of time-triggered traffic in time-sensitive networks. *IEEE Trans. Ind. Inf.* 16(7), 4525–4534 (2020)
- Finn, N.: Multiple cyclic queuing and forwarding. doi:<https://www.ieee802.org/1/files/public/docs2021/new-finn-multiple-CQF-0921-v02.pdf>. Accessed: 1 October 2021.
- Alexandris, K., Pop, P., Wang, T.: Configuration and evaluation of Multi-CQF shapers in IEEE 802.1 time-sensitive networking (TSN). *IEEE Access* 10, 109068–109081 (2022)
- Chen, L., Dorr, J., et al.: Use Cases – IEEE P802.1DG.[online] Available: <https://www.ieee802.org/1/files/public/docs2019/dg-pannell-automotive-use-cases-0919-v04.pdf>.
- Traffic Categories & Overall Performance Goals, Amrit Gopal, Ford Motor Company. doi:<https://www.ieee802.org/1/files/public/docs2022/dg-gopal-TrafficClassification-0222-v01.pdf>. Accessed: 1 December 2023.
- IEEE: IEEE Standard for Local and metropolitan area networks—Audio Video Bridging (AVB) Systems. IEEE, Piscataway, NJ (2011)

36. OMNeT++ Discrete Event Simulator. doi: <https://www.omnetpp.org>. Accessed: 1 October 2022. [Online].
37. Hackel, T., Meyer, P., Korf, F.: A simulation model for software-defined networking for communication over real-time Ethernet. In: Proceedings of the 6th International OMNeT++ Community Summit, pp. 1–10. EasyChair, Manchester (2019)
38. GUROBI OPTIMIZATION. doi:<https://www.gurobi.com/>. Accessed 10 October 2022.

How to cite this article: Nie, H., Su, Y., Zhao, W., Mu, J.: Hybrid traffic scheduling in time-sensitive networking for the support of automotive applications. *IET Commun.* 18, 111–128 (2024).
<https://doi.org/10.1049/cmu2.12713>